Cap Review

A smart contract assessment of Cap contracts was performed by Octane Security, with a focus on the security aspects of the application's implementation.

About Octane Security

Octane Security is an AI-enhanced blockchain security firm dedicated to safeguarding digital assets. We combine security research with cutting-edge machine learning models to maximize protocol security and minimize risk. Our team ranks in the top of audit contest leaderboards, captures bug bounties, and has served big-name clients such as Alchemy, Optimism, Notional Finance, Blast, and Graph Protocol in the past.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We also rely on information that is provided by the client, its affiliates and partners for vulnerability identification. We can not guarantee the absense of vulnerabilities, issues, flaws, or defects after the review. It is up to the client to decide whether to implement recommendations and we take no liability for invalid recommendations. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

Security Assessment Summary

The scope of this review was limited to files at commit: faff378edf4e6430d775397e21ee7dd6b243b769

Scope

The following smart contract was in scope of the audit:

- cap-contracts/contracts/delegation/Delegation.sol

Findings

Finding Number	Finding Title
H-1	Unadjusted Epoch-Boundary Timestamp In Delegateion.coverage() Causes Premature Liquidation And Reward Misrouting (FIXED)
L-1	Permissionless `distributeRewards` Allows Dust Attack, Causing Excessive Reward Claiming Overhead
I-1	Liquidation Threshold Can Be Effectively Set To 100% Due To LTV And Buffer Configuration
I-2	Missing Functionality To Remove Or Deregister Agents
I-3	Unused Oracle Storage Variable In Delegation Contract
I-4	Redundant Check In `slashTimestamp`
I-5	Incorrect Slashing Value Emitted In SlashNetwork Event

H-1 Unadjusted Epoch-Boundary Timestamp In Delegateion.coverage() Causes Premature Liquidation And Reward Misrouting

Description

The function `Delegation.coverage()` computes coverage using a minimum value that includes the Symbiotic slashable collateral evaluated at the **current epoch boundary timestamp**:

```
currentEpochStart = epoch() * epochDuration
```

At epoch-boundary blocks, `currentEpochStart` is equal to `block.timestamp`. In Symbiotic, the underlying slasher's view function (`slashableStake`) returns `0` when `captureTimestamp >= block.timestamp`. As a result, the slashable collateral term evaluates to `0` exactly at epoch boundaries.

Since `coverage()` returns the **minimum** across several terms, this causes `coverage()` to temporarily drop to `0` at epoch-boundary blocks, even if other coverage components are non-zero.

This behavior was introduced by adding an epoch-boundary slashable-collateral term **without applying the standard `-1` timestamp adjustment**, which is already used elsewhere in the codebase (e.g., `slashTimestamp`, `lastBorrowMinusOne`) to avoid boundary conditions.

This leads to multiple downstream issues:

1. **Reward misrouting**: When `coverage()` returns `0`, `Delegation.distributeRewards()` treats the agent as uncovered and routes all rewards to `feeRecipient` instead of the Symbiotic staker rewards contract.
2. **Premature liquidation**: `ViewLogic.agent` computes agent health based on `coverage()`. With `coverage() == 0` and outstanding debt, health becomes `0`, allowing liquidation to be opened immediately.
3. **Bypassing grace/expiry checks**: Liquidation can proceed through the emergency path, skipping grace and expiry checks, resulting in immediate slashing of restaker collateral.
4. **Temporary borrow denial (DoS)**: At epoch boundaries, `coverage() == 0` causes `maxBorrowable()` to return `0`, preventing new borrows during those blocks.

Recommendation

Apply a consistent **epoch-boundary timestamp adjustment** when querying slashable collateral, such as using:

```
epoch() * epochDuration - 1
```

This mirrors the defensive pattern already used elsewhere in the protocol and prevents coverage from dropping to zero at epoch boundaries.

Resolution

The issue is fixed in PR#232 by adjusting the current epoch start by subtracting one if it aligns with the current block timestamp.

L-1 Permissionless `distributeRewards` Allows Dust Attack, Causing Excessive Reward Claiming Overhead

Description

The function `Delegation.distributeRewards` has the following properties:

- It is **permissionless**, meaning anyone can call it.
- If the provided agent has **zero coverage**, the rewards are transferred to `$.feeRecipient`.
- If the provided agent has **nonzero coverage**, the rewards are forwarded to the `stakerRewarder` associated with that agent. This behavior can be seen in `SymbioticNetworkMiddleware.distributeRewards`:

```
function distributeRewards(address _agent, address _token) external
checkAccess(this.distributeRewards.selector) {
    SymbioticNetworkMiddlewareStorage storage $ =
getSymbioticNetworkMiddlewareStorage();
    uint256 _amount = IERC20(_token).balanceOf(address(this));

    address _vault = $.agentsToVault[_agent];
    address stakerRewarder = $.vaults[_vault].stakerRewarder;

    IERC20(_token).forceApprove(stakerRewarder, _amount);
    IStakerRewards(stakerRewarder)
        .distributeRewards(
            $.network, _token, _amount, abi.encode(uint48(block.timestamp - 1),
$.feeAllowed, "", "")
        );
}
```

Inside `DefaultStakerRewards.distributeRewards`, each forwarded reward is appended to the `RewardDistribution` array:

```
mapping(address token => mapping(address network => RewardDistribution[]
rewards_)) public rewards;
```

```
if (distributeAmount > 0) {
    rewards[token][network].push(
        RewardDistribution({amount: distributeAmount, timestamp: timestamp})
    );
}
```

When a shareholder later claims rewards, they are processed **sequentially**, in the exact order they were added to this array:

```
for (uint256 i; i < rewardsToClaim;) {
    RewardDistribution storage reward = rewardsByTokenNetwork[rewardIndex];

    amount += IVault(VAULT)
        .activeSharesOfAt(msg.sender, reward.timestamp, activeSharesOfHints[i])
        .mulDiv(reward.amount, _activeSharesCache[reward.timestamp]);

    unchecked {
        ++i;
        ++rewardIndex;
    }
}
```

To prevent unbounded loops and excessive gas usage, the claimant can specify a `maxRewards` value, which caps the number of reward entries processed in a single transaction:

```
uint256 rewardsToClaim =
    Math.min(maxRewards, rewardsByTokenNetwork.length - lastUnclaimedReward_);
```

Attack scenario

- Assume the reward token is **USDT**.
- An attacker deposits **1 wei of USDT** (the smallest unit).
- The attacker calls `Delegation.distributeRewards` with an agent that has **nonzero coverage**.
- As a result, a reward entry of **1 wei** is appended to the `RewardDistribution` array.
- The attacker repeats this process **1,000 times**, growing the array to 1,000 entries of dust rewards:

```
rewards[USDT][network] = [
    RewardDistribution({amount: 1, timestamp: t0}),
    RewardDistribution({amount: 1, timestamp: t1}),
    ...
    RewardDistribution({amount: 1, timestamp: t999})
];
```

- Later, a legitimate reward of **5,000 USDT** is distributed to the same agent, adding one more entry:

```
rewards[USDT][network] = [
    RewardDistribution({amount: 1, timestamp: t0}),
    ...
    RewardDistribution({amount: 1, timestamp: t999}),
    RewardDistribution({amount: 5000 * 10**6, timestamp: t1000})
];
```

- When a shareholder tries to claim rewards, they must process all **1,000 dust entries first** before reaching the real 5,000 USDT reward.
- If there are **200 shareholders**, this results in **200 × 1,000 transactions**, since each shareholder must individually process these dust rewards to eventually reach the meaningful reward.

In summary, the attacker only needs to submit a relatively small number of transactions to insert dust rewards, but this forces a much larger number of transactions on legitimate users, significantly increasing their gas costs and degrading the reward-claiming experience.

Recommendation

It is recommended to **either restrict `RewardDistribution` to authorized callers or prevent the distribution of rewards below a minimum threshold**.

I-1 Liquidation Threshold Can Be Effectively Set To 100% Due To LTV And Buffer Configuration

Description

During **adding or modifying an agent**, the protocol enforces that the **liquidation threshold** must be less than 100% and also less than **LTV + LTV buffer**:

```
if (_liquidationThreshold > 1e27) revert InvalidLiquidationThreshold();
if (_ltv != 0 && _liquidationThreshold < _ltv + $.ltvBuffer) revert
LiquidationThresholdTooCloseToLtv();
```

The issue arises because, depending on the values of `_ltv` and `$.ltvBuffer`, the **liquidation threshold can be set to 100%**. For example:

- `_ltv = 95%`
- `$.ltvBuffer = 5%`
- Resulting maximum `_liquidationThreshold = 100%`

A **100% liquidation threshold** is problematic because if the **borrowed amount equals the collateral**, the borrower **cannot be liquidated**.

- Example:
    - Collateral = $100
    - Borrowed = $100 → LTV = 100%
    - If borrowed value increases to $110, liquidators are **disincentivized**, because they would pay $110 to seize $100 collateral.

This issue is primarily caused by **misconfiguration** and can be mitigated by **restricting the maximum values of `_liquidationThreshold`, `_ltv`, and `$.ltvBuffer`** to ensure there is always a liquidation incentive.

Recommendation

It is recommended to **ensure that both `_ltv + $.ltvBuffer` and `liquidationThreshold` remain below 100%**.

I-2 Missing Functionality To Remove Or Deregister Agents

Description

The contract provides functions for **adding** and **modifying** agents, but there does not appear to be a corresponding function for **removing** an agent:

```
function addAgent(address _agent, address _network, uint256 _ltv, uint256
_liquidationThreshold)
    external
    checkAccess(this.addAgent.selector)
{
    // ...
}
```

```
function modifyAgent(address _agent, uint256 _ltv, uint256 _liquidationThreshold)
    external
    checkAccess(this.modifyAgent.selector)
{
    // ...
}
```

This suggests that once an agent is added, there is no explicit mechanism to fully remove or deregister it.

Recommendation

It is recommended to **add a dedicated function to support removing agents**.

I-3 Unused Oracle Storage Variable In Delegation Contract

Description

The `Delegation` contract initializes a storage variable named `oracle` during contract initialization:

```
function initialize(address _accessControl, address _oracle, uint256
_epochDuration) external initializer {
    __Access_init(_accessControl);
    __UUPSUpgradeable_init();
    DelegationStorage storage $ = getDelegationStorage();
    $.oracle = _oracle;
    $.epochDuration = _epochDuration;
    $.ltvBuffer = 0.05e27; // 5%
}
```

However, the `oracle` variable is **never referenced or used** anywhere else in the codebase. This makes the variable redundant in its current state and may cause confusion during audits or future maintenance, as it suggests oracle-based logic that does not actually exist.

Unused storage variables also increase the contract's state footprint without providing functional value.

Recommendation

Either:

- Remove the unused `oracle` variable entirely, or
- Clearly document it as a **placeholder for future upgrades**, explicitly stating its intended purpose to avoid ambiguity.

I-4 Redundant Check In `slashTimestamp`

Description

In the function `slashTimestamp`, the `_slashTimestamp` is computed as:

```
Math.max(
    (epoch() - 1) * $.epochDuration,
    $.agentData[_agent].lastBorrow > 0 ? $.agentData[_agent].lastBorrow - 1 : 0
)
```

- The first part, `(epoch() - 1) * $.epochDuration`, is always **less than** `block.timestamp`. This is because `epoch() = block.timestamp / $.epochDuration`, so `(epoch() - 1) * $.epochDuration` is approximately `block.timestamp - $.epochDuration`.
- The second part, `$.agentData[_agent].lastBorrow - 1`, is also **at most** `block.timestamp - 1`, since `$.agentData[_agent].lastBorrow` can never exceed `block.timestamp`.

Therefore, **`_slashTimestamp == block.timestamp` can never be true**, because both expressions used to compute `_slashTimestamp` are always smaller than the current block timestamp.

Recommendation

It is recommended to remove the check `if (_slashTimestamp == block.timestamp) _slashTimestamp -= 1;`.

I-5 Incorrect Slashing Value Emitted In SlashNetwork Event

Description

During the slashing process, the `SlashNetwork` event is emitted as follows:

```
function slash(address _agent, address _liquidator, uint256 _amount) external
checkAccess(this.slash.selector) {
    // ...
    emit SlashNetwork(network, _amount);
}
```

The issue is that the event emits the **requested slash amount** (`_amount`) rather than the **actual slashed share** (`slashShare`) that is computed and applied internally.

This is inconsistent with the event definition, which explicitly expects a value representing the slashed share:

```
event SlashNetwork(address network, uint256 slashShare);
```

As a result, off-chain systems, indexers, or monitoring tools that rely on this event may incorrectly calculate or display slashed amounts, leading to inaccurate reporting and analytics.

Recommendation

Emit the actual slashed share instead of the requested amount:

```
emit SlashNetwork(network, slashShare);
```